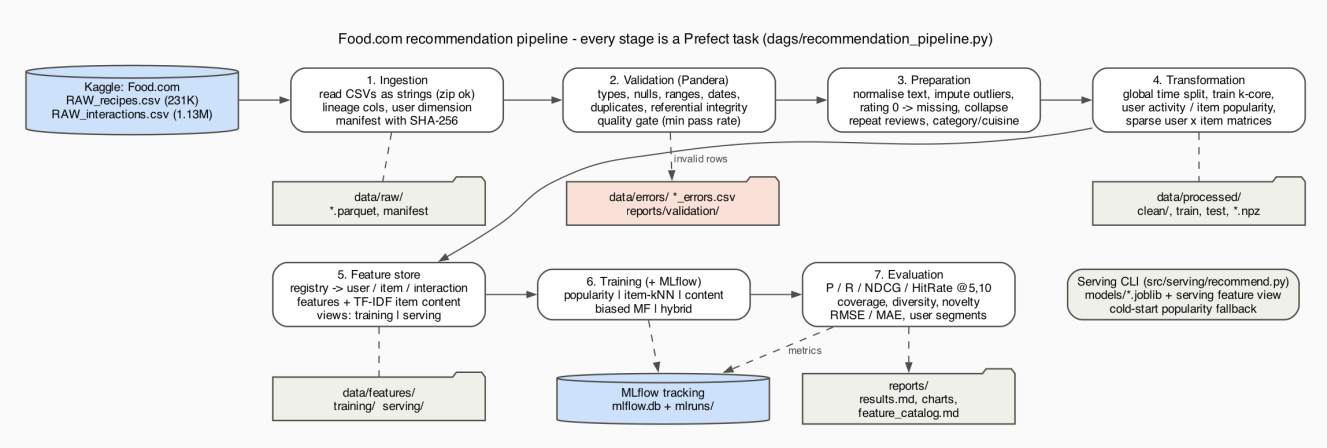
End-to-End Data Management Pipeline for a Recipe Recommender

DM4ML Assignment 2 · Yash Sharma · Roll No. 2025AG05572 · Repository: github.com/2025ag05572-gif/bitsdm4mlas2

1. Problem and data

I built a recommendation pipeline for a recipe website: given what a user has reviewed so far, the system ranks recipes they are likely to review (and rate highly) next. I used the public **Food.com** dataset from Kaggle (Majumder et al., 2019): 231,637 recipes (tags, ingredients, nutrition, prep time, steps) and 1,132,367 reviews from 2000–2018 with 0–5 star ratings. Food.com has no user profile table, so the ingestion stage reconstructs a user dimension (236,719 accounts) from everyone who wrote a review or uploaded a recipe. The data is very sparse and very positive: 76% of star ratings are 5, the mean is 4.66, and the median user has only one review.



2. Design decisions and tools

Ingestion. Both CSVs are read straight from the Kaggle zip *as strings*, so badly typed values get caught by validation instead of crashing the reader or being silently coerced. Every row gets lineage columns (`_row_id`, `_source_file`), and a manifest stores the SHA-256 of each source file, so any run can be traced back to the exact input. The data is stored as Parquet in `data/raw/`.

Validation (Pandera + custom checks). Each table is first type-coerced column by column (type errors are recorded), then checked against a Pandera schema (nulls, positive ids, allowed ratings 0–5, a date window of 1999–2018, valid list literals, 7-value nutrition), and finally passed through custom checks: exact duplicates, duplicate keys, referential integrity (review → recipe, review → user) and timestamp consistency. 72 rules are checked in total. Rows that break an *error* rule are written to `data/errors/` with the list of rules they broke. *Warnings* keep the row, and preparation handles them. A quality gate stops the Prefect flow if any table's pass rate falls below 95%. On the real data, the only error rule that fired was an invalid timestamp: **1,246 reviews are dated before their recipe was submitted**, and these were quarantined. Warnings included 4,979 missing descriptions, 1,094 recipes with 0 minutes, 70 with more than 30 days (one says 2,147,483,647 minutes), 125 with more than 10,000 calories, and 60,847 reviews without stars.

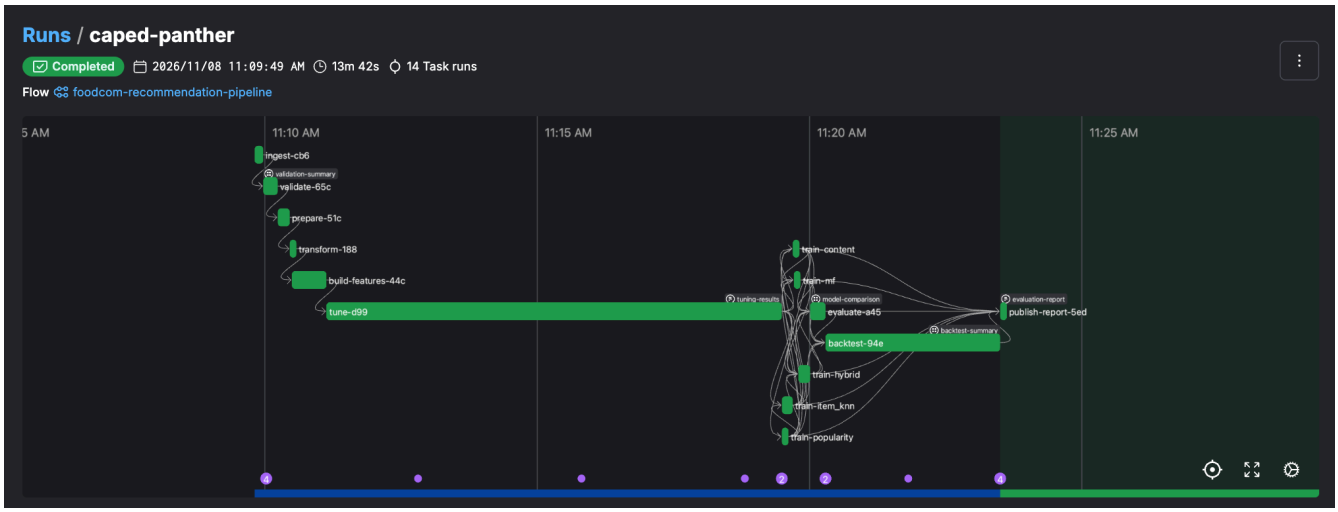
Preparation and transformation. Text and tags are normalised. Implausible minutes and calories are set to missing and imputed with the category median (1,164 and 125 recipes). A primary category and cuisine are derived from tags. Rating 0 means "review without stars", so I store it as a *missing rating* but keep the review as an implicit signal. I used a **global time-based split**: the latest 10% of reviews (from 26-Feb-2014) form the test set. The 5-core filter is applied to train only, giving 525,688 reviews from 16,865 users on 39,405 recipes (density 0.079%). Evaluation is warm-start: 2,235 test users with 11,453 relevant (≥4 stars) reviews. Derived datasets are a user activity summary, item popularity, and sparse user×item interaction, rating and review-date matrices, all computed from train only.

Feature management. Instead of Feast I wrote a small feature store. `registry.py` defines 36 documented features: 15 user features (activity count, average rating, days since last review, preferred category, category entropy, …), 15 item features (popularity, Bayesian rating, recent popularity, age, category, minutes, a TF-IDF content vector, …) and 6 interaction features. The same `compute_features(as_of)` code materialises a **training view** (as of the cutoff, so it is point-in-time correct) and a **serving view** (all data). `FeatureStore` is the only read path for training, evaluation and the `recommend.py` CLI, which falls back to recent popularity for unknown users. The catalog in `reports/feature_catalog.md` is generated from the registry.

Models and tracking. (1) A **popularity** baseline, using review counts from the feature store. (2) **Item-kNN** collaborative filtering: cosine similarity with shrinkage, down-weighting of heavy reviewers, and top-N neighbours. (3) **Content-based**: TF-IDF over tags and ingredients. (4) **Biased matrix factorisation**: mini-batch SGD on star ratings. (5) A **hybrid** that blends item-kNN, content and popularity, using the `user_activity_count` and `user_preferred_category` features. Hyperparameters are chosen by a **tuning stage that never touches the test set**. It carves a validation window (Aug-2011 to Feb-2014) out of the end of the training period, rebuilds features as of that inner cutoff, and logs 68 grid-search trials as nested MLflow runs. Every model run logs its parameters, data statistics, fit time, MF learning curve, model file, config and feature metadata to **MLflow**. Evaluation then logs its metrics and confidence intervals back to the same run.

Evaluation design. The main test set is all warm reviews after the cutoff. Each metric gets a **95% bootstrap confidence interval** over users (1,000 resamples). Every model is compared with the popularity baseline using a **paired bootstrap test** on the same users. As a second check, a **rolling-origin backtest** re-fits every model on 4 expanding windows and tests each on the following year (Feb-2014 to Feb-2018). All four test years come after the tuning window, so model selection never sees them.

Orchestration and reproducibility. `dags/recommendation_pipeline.py` is a **Prefect** flow with 14 tasks: ingest → validate → prepare → transform → features → tune → 5 × train → evaluate → backtest → publish. It has retries on ingestion, the quality gate, and table and markdown artifacts in the UI. `--serve` registers a weekly schedule. A full run takes about 14 minutes on a laptop (screenshot below). All settings live in one YAML file, and the run is reproducible through fixed seeds, pinned `requirements.txt`, source checksums, and a pipeline run id stored on every MLflow run. `make sample` runs the same code on a small synthetic sample in the same file format, and pytest covers the metrics, the bootstrap, the validation rules, the quality gate and the absence of leakage across the split.



3. Results

Test window 26-Feb-2014 to 20-Dec-2018, 2,235 warm users. Seen recipes are excluded. Coverage is the share of the 39,405 train recipes that appear in any top-10 list. Diversity is 1 – mean pairwise TF-IDF cosine similarity within a list. Δ is the paired difference in NDCG@10 against popularity, with its bootstrap p-value.

Model	P@10	R@10	NDCG@10 [95% CI]	Δ vs pop. (p)	Coverage@10	Diversity@10	RMSE	MAE
Popularity	0.0068	0.0246	0.0148 [0.0118, 0.0181]	–	0.0011	0.913	0.629	0.420
Item-kNN	0.0069	0.0238	**0.0164** [0.0130, 0.0202]	+0.0016 (0.11)	0.0062	0.905	–	–
Content (TF-IDF)	0.0006	0.0020	0.0013 [0.0004, 0.0024]	–0.0135 (<0.001)	**0.1028**	0.649	–	–
Biased MF	0.0001	0.0003	0.0003 [0.0000, 0.0007]	–0.0146 (<0.001)	0.0041	**0.957**	**0.586**	**0.360**
Hybrid	**0.0074**	**0.0251**	0.0160 [0.0129, 0.0193]	+0.0011 (**0.04**)	0.0012	0.906	–	–

Predicting the global mean rating gives RMSE 0.639 and MAE 0.437.

Rolling-origin backtest, NDCG@10 on each following year (users per year: 1,539 / 850 / 704 / 587):

Model	2014–15	2015–16	2016–17	2017–18	Mean ± SD	Years > popularity
Item-kNN	0.0144	0.0146	0.0102	0.0120	**0.0128 ± 0.0021**	3 / 4
Hybrid	0.0145	0.0134	0.0105	0.0108	0.0123 ± 0.0020	2 / 4
Popularity	0.0133	0.0127	0.0111	0.0114	0.0121 ± 0.0011	–
Content / MF	≤ 0.0014				0.0010 / 0.0005	0 / 4

- **Popularity is a very strong baseline, and the personalised gains are small.** Item-kNN and the hybrid both beat popularity on the main split. Only the hybrid's gain is statistically significant ($p = 0.04$), and the intervals of all three overlap. The backtest agrees: item-kNN wins in 3 of 4 years by about 6% on average, but popularity is the most stable of the three (lowest SD). So I would deploy item-kNN or the hybrid, with popularity as a fallback that is just as good.
- **Tuning moved item-kNN towards popularity.** Down-weighting heavy reviewers (some have more than 2,000 reviews) together with stronger shrinkage raised validation NDCG@10 from about 0.011 to 0.020. It also cut coverage from 22% to 1%, because both settings favour well-reviewed recipes. That is an accuracy–coverage trade-off a real product would have to decide on explicitly.
- **Matrix factorisation is good at rating prediction but useless for ranking.** It has the lowest RMSE (8% below the global mean), but because almost every rating is 4–5 stars, its item biases mostly pick obscure recipes with a couple of 5-star reviews. This is the well-known gap between RMSE and top-N accuracy. Content-based filtering has the highest coverage but low accuracy, because many tags are generic (e.g. "60-minutes-or-less").

4. Limitations and future work

- **Ratings and relevance are close to "reviewed".** Ratings are so positive that "relevant = 4+ stars" means almost any review. Treating star-less (0) reviews as missing is my own interpretation. An implicit-feedback model (ALS or BPR) would probably rank better than explicit MF.
- **User data is thin.** The reconstructed user table has no demographics, so user features come only from behaviour.
- **Infrastructure is local only.** The feature store is offline Parquet with full recomputation (no online store or incremental updates), MLflow and Prefect run locally, and data versioning relies on checksums rather than a tool like DVC. Adding Feast or another online store, DVC, and a scheduled Prefect deployment with drift monitoring would be the next steps.